INTERN TRAINING PROGRAM

25-Day Python Mastery Plan

A structured 5-week training roadmap for Python interns — from fundamentals to industry-grade backend development.

25	5	25
TRAINING DAYS	WEEKS	HANDS-ON TASKS

5

MINI PROJECTS

Python · MySQL · PostgreSQL · SQLAlchemy · FastAPI · JWT Auth · Redis · Celery · SMTP · Pandas · pytest · Docker-ready

One task per day · Mon–Fri schedule · Capstone on Day 25

Table of Contents

23 Redis + Caching

24 Celery + Cron Jobs

25 Capstone Project

25-Day Roadmap Overview

Week	Theme	Days	Key Skills
Week 1	**Python Foundations**	1 – 5	Basics Collections OOP Comprehensions Functional Tools
Week 2	**Core Python Tools**	6 – 10	File I/O JSON Logging Regex venv
Week 3	**Advanced Python**	11 – 15	Decorators SMTP REST APIs Threads pytest
Week 4	**Databases**	16 – 20	MySQL PostgreSQL SQLAlchemy Pandas Matplotlib
Week 5	**Industry Real-World**	21 – 25	FastAPI JWT Redis Celery Capstone

Industry Stack Covered

Tool / Library	Industry Use Case
MySQL	Transactional DBs — e-commerce, CMS, user management
PostgreSQL	Enterprise, fintech, analytics, data-heavy systems
SQLAlchemy ORM	Standard ORM in Python backends — Flask, FastAPI, standalone
FastAPI	Modern REST APIs, microservices, SaaS backends
JWT Auth	Securing APIs — used in every microservice and mobile backend
Redis	Caching, sessions, pub/sub — reduces DB load by 80–90%
Celery	Background jobs — async emails, report generation, nightly tasks
SMTP	Automated email notifications, alerts, report delivery

Tool / Library	Industry Use Case
python-dotenv	Secrets management — mandatory in all production apps
pytest	Automated testing — part of every CI/CD pipeline

WEEK 01 / 05

Python Foundations

Refresh Python basics and build strong intermediate skills with Collections, OOP, Comprehensions, and Functional Tools.

Day 1 — Basics Refresh
Day 2 — Collections + Generators
Day 3 — OOP Part 1
Day 4 — OOP Part 2
Day 5 — Mini Project

DAY 01 Python Basics Refresh

TOPICS

- Variables, data types (int, float, str, bool)
- Strings — indexing, slicing, methods (upper, lower, split, replace, strip)
- Lists, Tuples, Sets, Dictionaries — CRUD
- Loops (for, while), conditionals (if/elif/else)
- Functions — def, return, default args, *args, **kwargs

DAILY TASK

Take a list of 10 employee names and salaries as a dictionary. Print:

- ✓ Top 3 highest earners
- ✓ Average salary
- ✓ Count of employees above/below average
- ✓ All names sorted alphabetically in uppercase

DAY 02 Collections Module + Comprehensions & Generators

TOPICS

- **Collections:** Counter, defaultdict, deque, namedtuple
- List, dict, set comprehensions with conditions
- Nested comprehensions
- Generator function with `yield`
- Generator expression vs list (memory difference)

DAILY TASK

- ✓ Use Counter to find top 3 frequent words in a paragraph
- ✓ Use defaultdict to group employees by department
- ✓ List comprehension — filter salaries above 50000 and square them
- ✓ Write a generator that yields fibonacci numbers — print first 10

DAY 03 OOP Part 1 — Classes & Properties

TOPICS

- Class, `__init__`, `self`
- Instance variables vs class variables
- `__str__` and `__repr__`
- `@property` and `@property.setter`
- `@classmethod` and `@staticmethod`

DAILY TASK

Build a **BankAccount** class:

- ✓ Owner name and balance as instance variables
- ✓ deposit() and withdraw() methods (block insufficient funds)
- ✓ @property for read-only balance
- ✓ @classmethod to create account from a dict
- ✓ Clean summary using __str__

DAY 04 OOP Part 2 + Functional Tools

TOPICS

- Inheritance and `super()`
- Method overriding and polymorphism
- Encapsulation (_ prefix convention)
- `isinstance()`, `issubclass()`
- map(), filter(), zip(), lambda, sorted(key=)

DAILY TASK

- ✓ Base class Employee → inherit Manager and Intern
- ✓ Each overrides get_details() with different output
- ✓ Store all in a list, call get_details() polymorphically
- ✓ Use filter + lambda to get employees with salary > 40000
- ✓ Use sorted(key=) to rank by salary and print leaderboard

DAY 05 Week 1 Mini Project — Contact Book

MINI PROJECT

- ✓ Contact class (name, phone, email) with __str__
- ✓ ContactBook class: add, search, delete, list all contacts
- ✓ Use defaultdict to group contacts by first letter (A–Z)
- ✓ Use filter + lambda for search functionality
- ✓ Menu-driven console interface (loop until exit)
- ✓ Handle all edge cases (duplicate, not found, empty book)

WEEK 02 / 05

Core Python Tools

Master essential Python tools used in every real project — file handling, JSON, logging, regex, and package management.

Day 6 — File + Exceptions | Day 7 — JSON + Logging | Day 8 — Regex | Day 9 — Modules + venv | Day 10 — Mini Project

DAY 06 File Handling + Exception Handling

TOPICS

- open(), read, write, append — with statement
- csv module: DictReader, DictWriter
- try / except / else / finally
- Common exceptions: ValueError, TypeError, FileNotFoundError
- Raising custom exceptions with raise

DAILY TASK

- ✓ Read a CSV of employees (name, department, salary)
- ✓ Find average salary per department
- ✓ Write new CSV with "Bonus" column (10% of salary)
- ✓ Raise custom EmptyFileError if file is empty
- ✓ Handle all bad rows gracefully with error messages

DAY 07 JSON Handling + Logging

TOPICS

- json: loads, dumps, load, dump, indent=
- Nested JSON parsing, .get() for missing keys
- logging: DEBUG, INFO, WARNING, ERROR, CRITICAL
- Log to file + console simultaneously with timestamps
- RotatingFileHandler — control log file size

DAILY TASK

- ✓ Read students.json with nested subject marks
- ✓ Calculate total, average, pass/fail per student
- ✓ Add "result" key to each record
- ✓ Write updated data to output.json with clean formatting
- ✓ Log every step (reads, writes, errors) to app.log + console

DAY 08 Regular Expressions

TOPICS

- re module: match, search, findall, sub, split
- Patterns: \d, \w, \s, [], ^, $, +, *, ?
- Groups and named groups
- re.compile() for reusable patterns

DAILY TASK

Write regex functions to:

- ✓ Validate email address format
- ✓ Validate Indian phone number (10 digits)
- ✓ Extract all dates (DD/MM/YYYY) from a paragraph
- ✓ Mask credit card numbers — show only last 4 digits

DAY 09

Modules, Packages & Virtual Environments

TOPICS

- Creating custom modules and importing them
- `__name__ == "__main__"` pattern
- venv: create, activate, deactivate
- pip: install, freeze, requirements.txt
- Standard library: os, sys, datetime, random, math

DAILY TASK

- ✓ Create utils.py with helper functions (age from DOB, random password generator, file size checker)
- ✓ Import and use in main.py
- ✓ Set up a virtual environment
- ✓ Install requests + rich packages
- ✓ Generate requirements.txt with pip freeze

DAY 10

Week 2 Mini Project — Student Report Generator

MINI PROJECT

- ✓ Read student data from students.json
- ✓ Validate email and phone fields using regex
- ✓ Calculate grades (A/B/C/F) and class ranks
- ✓ Write final report to report.json and report.csv
- ✓ Log every step to report.log with timestamps
- ✓ Custom exception classes for invalid data and empty files

WEEK 03 / 05

Advanced Python

Level up with decorators, itertools, email automation, API integration, multithreading, and professional testing.

Day 11 — Decorators + Itertools

Day 12 — SMTP

Day 13 — APIs

Day 14 — Threads + pytest

Day 15 — Mini Project

DAY 11 Decorators & Closures + Itertools & Functools

TOPICS

- Closures — function remembering outer scope
- Writing decorators with @, chaining, functools.wraps
- itertools: combinations, groupby, islice, chain, product
- functools: reduce, partial, lru_cache

DAILY TASK

- ✓ Write @timer (execution time) and @logger (function name + args) decorators
- ✓ Use lru_cache on fibonacci — compare time with @timer
- ✓ Use combinations to generate team fixtures from 5 teams
- ✓ Use groupby to group transactions by category

DAY 12 SMTP — Sending Emails

TOPICS

- smtplib: SMTP_SSL, plain text emails
- HTML emails with MIMEMultipart + MIMEText
- File attachments with MIMEBase + encoders
- Gmail SMTP with App Password setup
- Sending to multiple recipients
- Store credentials in .env (never hardcode)

DAILY TASK

- ✓ Send a plain text email via Gmail SMTP
- ✓ Send an HTML styled report email
- ✓ Attach report.csv from Day 10 as file attachment
- ✓ Store SMTP credentials in .env using python-dotenv
- ✓ Log sent/failed status per recipient

DAY 13 Working with APIs (requests library)

TOPICS

- requests: GET, POST, PUT, DELETE
- Headers, query params, request body, timeout
- Parsing JSON responses
- HTTP status codes and error handling
- Session objects for multiple requests

DAILY TASK

Using jsonplaceholder.typicode.com/users:

- ✓ GET all users, filter by city
- ✓ POST a new user record
- ✓ PUT to update an existing user
- ✓ DELETE one record
- ✓ Save final results to api_results.json
- ✓ Log all requests and responses with timestamps

DAY 14 Multithreading + Testing with pytest

TOPICS

- threading.Lock for shared resource safety
- ThreadPoolExecutor for concurrent tasks
- GIL concept — threads vs processes
- pytest: fixtures, parametrize, assertions
- unittest.mock for mocking external calls

DAILY TASK

- ✓ Fetch 5 URLs sequentially — measure time
- ✓ Repeat with ThreadPoolExecutor — compare and print time difference
- ✓ Use Lock to safely write results to shared log file
- ✓ Write pytest suite for BankAccount (Day 3) with parametrize
- ✓ Mock an external API call and test the function that uses it

DAY 15 Week 3 Mini Project — Daily Sales Report Automation

MINI PROJECT

- ✓ Read sales data from a JSON file
- ✓ Calculate: total sales, top 5 products, department summary
- ✓ Generate an HTML formatted report
- ✓ Send HTML email + CSV attachment via SMTP to multiple recipients
- ✓ Use ThreadPoolExecutor to send emails concurrently
- ✓ Decorate key functions with @timer and @logger decorators
- ✓ Log everything to automation.log with RotatingFileHandler

WEEK 04 / 05

Databases

Work with industry-standard databases — MySQL, PostgreSQL, SQLAlchemy ORM — and process data with Pandas and Matplotlib.

Day 16 — MySQL
Day 17 — PostgreSQL
Day 18 — SQLAlchemy ORM
Day 19 — Pandas + Visualization
Day 20 — Mini Project

DAY 16

MySQL with Python

Real-World Use: E-commerce order management, user records, inventory systems, CMS platforms.

TOPICS

- mysql-connector-python / pymysql
- Connect, create DB, create tables
- CRUD: INSERT, SELECT, UPDATE, DELETE
- Parameterized queries (prevent SQL injection)
- .env for DB credentials (python-dotenv)

DAILY TASK

- ✓ Connect Python to local MySQL DB
- ✓ Create employees table with proper schema
- ✓ Functions: add employee, get by department, update salary, delete by ID
- ✓ Use only parameterized queries (no string formatting in SQL)
- ✓ Store all credentials in .env file
- ✓ Log all DB operations to db.log

```
import mysql.connector from dotenv
import load_dotenv import os
load_dotenv() conn =
mysql.connector.connect(
host=os.getenv("DB_HOST"),
user=os.getenv("DB_USER"),
password=os.getenv("DB_PASSWORD"),
database=os.getenv("DB_NAME") )
```

DAY 17

PostgreSQL with Python

Real-World Use: Fintech, enterprise applications, analytics platforms, data-heavy systems requiring ACID compliance.

TOPICS

- psycopg2: connect, cursor, execute
- CRUD operations
- Transactions: commit(), rollback()
- PostgreSQL types: SERIAL, JSONB, TIMESTAMP
- Key differences vs MySQL in practice

DAILY TASK

- ✓ Connect to PostgreSQL, create orders table (SERIAL, TIMESTAMP)
- ✓ Functions: place order, get by customer, update status, delete
- ✓ Use transactions — rollback on any failure
- ✓ Store JSONB metadata field for each order
- ✓ Store credentials in .env

```
import psycopg2 conn =
psycopg2.connect(
host=os.getenv("PG_HOST"),
database=os.getenv("PG_DB"),
user=os.getenv("PG_USER"),
password=os.getenv("PG_PASSWORD")
)
```

DAY 18 SQLAlchemy ORM

Real-World Use: Industry standard ORM in Python backends — Flask-SQLAlchemy, FastAPI + SQLAlchemy. Raw SQL is rarely written directly in production code.

TOPICS

- What is ORM and why industry uses it
- Engine, Session, declarative_base
- Defining models as Python classes
- CRUD via ORM (no raw SQL)
- ForeignKey, relationship() between models
- Works with both MySQL and PostgreSQL

DAILY TASK

- ✓ Define Department and Employee ORM models with ForeignKey
- ✓ Write ORM functions: add, list, update, delete
- ✓ Query all employees in a department using relationship()
- ✓ Switch DB URL from MySQL to PostgreSQL — notice no code changes needed

```
class Employee(Base):
__tablename__ = "employees" id =
Column(Integer, primary_key=True)
name = Column(String(100)) dept_id
= Column(Integer,
ForeignKey("departments.id"))
department =
relationship("Department")
```

DAY 19 Pandas + Data Visualization

Real-World Use: ETL pipelines, HR analytics, sales dashboards, data cleaning before DB inserts, business intelligence reports.

TOPICS

- DataFrame CRUD, read_csv, read_json, to_csv
- groupby, merge, pivot_table, apply
- Handling nulls: fillna, dropna
- matplotlib + seaborn: bar, histogram, scatter, heatmap
- Saving charts as PNG

DAILY TASK

- ✓ Load employee data from PostgreSQL via SQLAlchemy into Pandas DataFrame
- ✓ Find average salary by department, top 5 earners
- ✓ Fill missing salary values with dept median
- ✓ Generate 2×2 dashboard: salary distribution, headcount by dept, salary vs experience scatter, correlation heatmap
- ✓ Save as hr_dashboard.png

DAY 20

Week 4 Mini Project — Employee Management CLI + DB

MINI PROJECT

- ✓ PostgreSQL DB via SQLAlchemy ORM (departments + employees tables)
- ✓ CRUD operations from command-line menu
- ✓ Bulk import employees from CSV using Pandas
- ✓ Generate department salary report (Pandas groupby)
- ✓ Save report as JSON file + PNG bar chart
- ✓ Load all DB credentials from .env
- ✓ Log all operations to app.log

WEEK 05 / 05

Real-World Industry

Build production-grade systems with FastAPI, JWT Authentication, Redis caching, and Celery background jobs — the full modern Python backend stack.

Day 21 — FastAPI Part 1

Day 22 — FastAPI + JWT

Day 23 — Redis + Caching

Day 24 — Celery + Cron

Day 25 — Capstone

DAY 21

FastAPI Part 1 + Environment Variables

Real-World Use: FastAPI is now the most adopted Python API framework in industry — used for microservices, SaaS backends, and internal tools.

TOPICS

- python-dotenv: .env, load_dotenv(), os.getenv()
- Why .env is mandatory in production (never hardcode)
- FastAPI: GET, POST routes, path & query params
- Request body with pydantic BaseModel
- Swagger UI at /docs (auto-generated)
- Connect FastAPI to PostgreSQL via SQLAlchemy

DAILY TASK

Build Employee Management API connected to PostgreSQL:

- ✓ GET /employees — list all employees
- ✓ GET /employees/{id} — get one employee
- ✓ POST /employees — add new employee
- ✓ All credentials from .env file
- ✓ Test all routes on http://127.0.0.1:8000/docs
- ✓ Log every request with timestamp

DAY 22

FastAPI Part 2 + JWT Authentication

Real-World Use: JWT is the industry standard for securing REST APIs — used in every microservice architecture, mobile app backend, and SaaS platform.

TOPICS

- PUT and DELETE routes
- JWT (JSON Web Token) — how it works
- python-jose for JWT encoding/decoding
- passlib[bcrypt] for password hashing
- Login route returning access token
- Protected routes with Depends() + token verification

DAILY TASK

Extend Employee API with auth:

- ✓ PUT /employees/{id} — update (JWT protected)
- ✓ DELETE /employees/{id} — delete (JWT protected)
- ✓ POST /auth/register — register user with hashed password
- ✓ POST /auth/login — returns JWT access token
- ✓ Return 401 Unauthorized for invalid/missing tokens

```
from jose import jwt from
passlib.context import
CryptContext pwd_context =
CryptContext(schemes=["bcrypt"])
SECRET_KEY =
os.getenv("SECRET_KEY")
```

DAY 23 **Redis + Caching**

Real-World Use: Redis is used in virtually every large-scale system. Caching GET endpoints reduces DB load by 80–90% and dramatically improves API response times.

TOPICS

- What is Redis and why caching matters in production
- redis-py: set, get, delete, expire (TTL)
- Caching API responses to reduce DB load
- Cache invalidation strategy on data change
- Storing JWT session data in Redis

DAILY TASK

- ✓ Cache GET /employees response for 60 seconds in Redis
- ✓ Cache GET /employees/{id} with TTL per record
- ✓ Invalidate relevant cache keys on every PUT/DELETE
- ✓ Store JWT session in Redis after successful login
- ✓ Log "CACHE HIT" vs "CACHE MISS" for every GET request

```
r =
redis.Redis(host=os.getenv("REDIS_HOST"),
port=6379) cached = r.get(f"employee:
{id}") if cached: return
json.loads(cached) # cache hit
r.setex(f"employee:{id}", 60,
json.dumps(data))
```

DAY 24

Celery + Cron Jobs

Real-World Use: Celery is the industry standard for background tasks in Python — async emails, report generation, file processing, nightly exports. Used in every production Django/FastAPI app at scale.

TOPICS

- What is Celery (distributed task queue)
- Celery with Redis as message broker
- @app.task — defining async background tasks
- .delay() and .apply_async() to trigger tasks
- celery beat — scheduled periodic tasks
- Flower dashboard for task monitoring

DAILY TASK

- ✓ Async task: send welcome email when new employee is added via POST /employees
- ✓ Scheduled task (celery beat): every 2 mins generate dept summary → save to DB + log
- ✓ Daily midnight task: export all employees to JSON backup file
- ✓ Monitor all tasks on Flower at http://localhost:5555

```
celery_app = Celery("tasks", broker=os.getenv("REDIS_URL"))
@celery_app.task def send_welcome_email(email, name): # SMTP logic from Day 12 pass
```

DAY 25

Capstone Project — HR Management System Backend

```
HR Management System — Production-Grade Backend
──────────────────────────────────────────── PostgreSQL + SQLAlchemy ORM └──
departments, employees, users, audit_logs FastAPI REST API ├── POST
/auth/register & POST /auth/login (JWT) ├── GET /employees (Redis cached, 60s
TTL) ├── POST /employees (triggers Celery welcome email) ├── PUT /employees/{id}
(JWT protected) ├── DELETE /employees/{id} (JWT protected) └── GET
/reports/summary Redis → Cache all GET routes + JWT sessions Celery → Async
welcome email + daily JSON backup Pandas → Salary analytics loaded from DB
Matplotlib → HR dashboard PNG auto-generated SMTP → Monthly report email + chart
attachment Logging → All events to rotating app.log .env → ALL credentials (never
hardcoded) pytest → Tests for all core API routes
```

Database Layer

PostgreSQL with SQLAlchemy ORM. Four tables with relationships. All queries via ORM — no raw SQL.

API Layer

FastAPI with full CRUD. JWT protected routes. Pydantic validation on all inputs. Swagger UI auto-docs.

Caching Layer

Redis caches all read endpoints. Cache invalidated on writes. JWT sessions stored in Redis.

Background Jobs

Celery worker sends welcome emails async. Celery beat runs scheduled reports and nightly backups.

Reporting

Pandas loads DB data for analytics. Matplotlib generates dashboard PNG. SMTP delivers monthly HTML report with attachment.

Quality

Full logging with rotation. All secrets in .env. pytest suite covering all API routes and business logic.

Environment Setup

Run once at the start of the training program. All libraries needed for all 25 days.

```
# Install all required libraries pip install mysql-connector-python psycopg2-
binary sqlalchemy \ fastapi uvicorn python-jose passlib[bcrypt] \ redis celery
flower python-dotenv \ pandas matplotlib seaborn requests pytest pydantic #
Sample .env file structure DB_HOST=localhost DB_USER=root
DB_PASSWORD=yourpassword DB_NAME=training_db PG_HOST=localhost PG_DB=training_pg
PG_USER=postgres PG_PASSWORD=yourpassword REDIS_HOST=localhost
REDIS_URL=redis://localhost:6379/0 SECRET_KEY=your-secret-key-here
SMTP_EMAIL=youremail@gmail.com SMTP_PASSWORD=your-app-password # Run FastAPI
server uvicorn main:app --reload # Run Celery worker celery -A tasks worker --
loglevel=info # Run Celery beat (scheduler) celery -A tasks beat --loglevel=info
# Run Flower (task monitor) celery -A tasks flower --port=5555
```

Golden Rules for the Intern

📌	**Never hardcode credentials** — always use .env and python-dotenv
📌	**Always use parameterized queries** — never format SQL strings with user input
📌	**Log, don't print** — replace all print() with proper logging in real projects
📌	**Handle exceptions** — never let a program crash without a meaningful error message
📌	**One task per day** — read 30 min → code the task → break something → fix it
📌	**The fixing is where learning happens** — errors are not failures, they are lessons

25-Day Python Intern Training Plan · 5 Weeks × 5 Days · One Task Daily